

INSERT
INTO department
(department_id, name, location_id) → Buena práctica aclarar las columnas, por si agregan nuevas values
(99, 'VENTAS', 122) → La PK (department_id) no puede ser nula

Errores posibles del INSERT

1. PK duplicada
 2. Algún campo no que se quiere ingresar, en la tabla original no existe
 3. Campo mayor al que soporte
-

UPDATE department
SET name='VENTAS 2'
WHERE department_id = 99; → Si no aclaramos condición, le modifica a todos los

UPDATE department
SET name='VENTAS 2'
WHERE department_id = 97; → No va a dar error por no existir el id, sino que va a devolver que no va a modificar ningún archivo

UPDATE department
SET name='VENTAS 2'
location_id = 128 → No va a dar error por no existir el id, sino que va a devolver que no va a
WHERE department_id = 97; modificar ningún archivo

Pasos que hace el UPDATE

1. Busca aquellos archivos que cumplan la condición (WHERE)
2. Ejecuta la actualización sobre esos archivos

Errores posibles del UPDATE

1. PK
 2. FK
-

DELETE
FROM department
WHERE department_id > 90;

Pasos que hace el DELETE ¡IGUAL QUE EL UPDATE!

1. Busca aquellos archivos que cumplan la condición (WHERE)
2. Ejecuta la actualización sobre esos archivos

Errores posibles del UPDATE

1. Error de FK: Intentar borrar una PK que es FK para otro archivo
2. Error de PK

Baja física	Baja lógica
Eliminar físicamente el registro. Puede generar eliminación en cascada ¡peligro!	Registro queda físicamente en la tabla, pero lógicamente se dió de baja. Por ejemplo, agregando campos "FI" (fecha inicio) y "FF" (fecha final) en empleados. Si FF es null, sigue trabajando.

- **COMMIT:** Se guardan los cambios realizados en la Base de Datos, definitivamente.
- **ROLLBACK:** Se eliminan los cambios realizados hasta el último COMMIT hecho sobre la Base de Datos.

Ver Restricciones en Oracle SQL Developer:

- Oracle Conexiones > DB_SALES > Tablas (Filtrado) > EMPLOYEE ← Hacemos click

Se abre la ventana "EMPLOYEE", y elegimos la columna "Restricciones" ✓

↪ Si violamos alguna de estas Restricciones, va a dar error!

Bloques Anónimos

No tienen nombre, por lo que no se pueden llamar.

Estructura:

- **DECLARE** → Sección Optativa

Variables;	siempre arranca con v_ y terminan en ;
Constantes;	siempre arranca con c_ y se le asigna el tipo CONSTANT + tipo

- **BEGIN**

Instrucciones

- **EXCEPTION** → Sección Optativa

Excepciones

- **END;**

- Ejemplos -

DECLARE

v_nombre varchar2(15):= 'Juan'; → Ya la inicialice con un valor

v_edad number(2);

v_fecha date;

v_precio number(6,2);

c_iva **CONSTANT** number(3,2):= 0.21; → Constante para calcular los precios finales con IVA

v_nombre2 **employee.first_name%type**; → **v_nombre2** se declara como el mismo tipo de **first_name**

BEGIN

v_nombre := 'Pepe'; → A la variable nombre le asigné Pepe

v_edad := &ingrese_su_edad; → Ingresar valor a asignar por teclado

v_nombre := '&ingrese_su_edad'; → Ingresar valor a asignar por teclado

dbms_output.put_line('Hola Mundo'); → Mostrarle información al usuario

dbms_output.put_line('Nombre: '||v_nombre); → Mostrar variable por consola

v_fecha := to_date('2012206', 'ddmmyyyy');

dbms_output.put_line('Vacaciones: '||v_fecha);

v_precio := 100;

dbms_output.put_line('Precio final: '||(v_precio+v_precio*c_iva)); → ¡Cuentas entre paréntesis!

END;

> **SET SERVEROUTPUT ON** → Antes de todo, hay que activar el server output para poder usar **dbms_output.put_line** e imprimir mensajes por consola al usuario!!!

Ciclo IF		
IF < cond > THEN - - - - - - - END IF;	IF < cond > THEN - - - ELSE - - - END IF;	IF < cond1 > THEN - ELSIF < cond2 > THEN - ELSIF < cond3 > THEN - ELSE → Optativo - END IF;

SELECT campo1, campo2
INTO variable1, variable2 → Los tipos de las variables tienen que ser iguales a los campos
FROM tabla
WHERE condición